

Servidor TCP concurrente en Java

Redes de Comunicaciones - Curso 2025/26

Introducción

En este boletín ilustraremos, con un ejemplo muy sencillo, cómo programar en Java un servidor TCP que atienda las conexiones de sus **clientes en segundo plano y de forma concurrente**. Para este fin utilizaremos las clases para **sockets TCP** que ya abordamos en el boletín anterior, y también recurriremos a la clase **Thread** de Java.

Parte de los contenidos de este boletín, en concreto los relacionados con los *threads* y el acceso concurrente a recursos compartidos, ya están cubiertos por la asignatura *Programación Concurrente y Distribuida*. Por ello, este boletín sólo pretende repasar de forma concisa los aspectos esenciales necesarios para el desarrollo del proyecto de prácticas *NanoFiles*.

Creación de hilos en un servidor concurrente

Un **servidor TCP concurrente debe estar preparado para aceptar**, en cualquier momento, **una petición de conexión formulada desde un nuevo host cliente**, independientemente de si en ese momento el servidor aún está procesando otras peticiones anteriores. Esto hace necesario que **exista un proceso en primer plano**, o alternativamente un **hilo principal en segundo plano**, que se dedique exclusivamente a escuchar las peticiones de conexión entrantes y crear un nuevo **hilo secundario** para cada nueva conexión aceptada.

La aceptación de peticiones de conexión la lleva a cabo, como es conocido, **el método `accept` de la clase `ServerSocket`**. Este método es una función síncrona que solamente "despierta" cuando llega al puerto de escucha una nueva petición de conexión. En cuanto al hilo secundario que se crea con cada nueva conexión, es imprescindible **que reciba el socket de la conexión (devuelto por `accept`)**. El hilo también debería tener **acceso a las estructuras de datos del servidor que sean compartidas por todos los hilos secundarios**, y que determinan el estado del servidor.

```
while (true) {  
    Socket socket = serverSocket.accept();  
    TCPServerThread st = new TCPServerThread(socket, state);  
    st.start();  
}
```

El nuevo hilo secundario, por **ser un hilo, deberá pertenecer a una clase descendiente de `Thread`**. Esto supone que en la declaración de la clase será necesario utilizar **`extends Thread`**, y además será preciso implementar el método **`run`**, que se **pondrá en marcha automáticamente cuando desde fuera se ejecute el método `start` del hilo**.

```
public class TCPServerThread extends Thread {

    private Socket socket;
    private TCPServerState serverState;

    public void run() {
        [...]
        DataOutputStream dos = new DataOutputStream(socket.getOutputStream());
        DataInputStream dis = new DataInputStream(socket.getInputStream());
        [...]

        // Update server *GLOBAL* state (shared by all threads)
        int reqNum = serverState.fetchAndIncrementNumRequests();
        [...]
    }

}
```

A partir de este momento, la nueva conexión TCP entre el servidor y el cliente recién conectado será **gestionada íntegramente por el hilo secundario**, apoyándose en los dos manejadores de flujo habituales, de las clases `DataInputStream` y `DataOutputStream`, respectivamente.

Este hilo permanecerá "vivo" únicamente mientras exista la conexión. Por lo tanto, **en el servidor siempre habrá tantos hilos secundarios como conexiones abiertas haya en ese momento**.

Manejo de estructuras de datos compartidas

Tal y como se ha apuntado anteriormente, los hilos secundarios que se estén ejecutando concurrentemente pueden necesitar compartir una o varias estructuras de datos que establezcan el estado del servidor. En consecuencia, cabe pensar en la posibilidad de que varios de estos **hilos pretendan acceder al mismo tiempo a las variables que dan soporte a estas estructuras de datos**. Si el acceso a las variables compartidas no se sincroniza adecuadamente, tendremos una condición de carrera y comportamientos inesperados en el programa. Esta situación se puede prevenir **con el uso de métodos *sincronizados* para leer o escribir en variables compartidas**, ya que un método `synchronized` **no puede ser ejecutado simultáneamente por varios hilos**.

```
public class TCPServerState {
    private int nextRequestNumber = 0;
    public synchronized int fetchAndIncrementNumRequests() {
        return ++nextRequestNumber;
    }
}
```

Ejemplo de servidor TCP concurrente

Los alumnos tienen a su disposición en el Aula Virtual un ejemplo que ilustra cómo construir un servidor TCP concurrente, el cual se encuentra listo para importar en un proyecto Eclipse llamado

`boletinTCPconcurrente`. A continuación, se describe dicho código de ejemplo.

`TCPServer.java` : proceso principal

En el servidor TCP el método `accept` de la instancia `serverSocket` se encuentra radicado en el interior de un bucle sin condición de finalización. **Previamente se ha creado el objeto `state`, concebido para guardar el estado del servidor, al que deberán poder acceder todos los hilos.**

Cuando `accept` despierta tras recibir una petición en el puerto `6969`, se crea el socket correspondiente, `socket`, e inmediatamente después se crea el hilo, `st`, que se pone en marcha con `start`. Este hilo recibe, a través de su constructor, el socket de conexión (`socket`) y el estado del servidor (`state`).

`TCPServerThread.java` : hilos del servidor para atender a cada cliente conectado

Cada hilo está asociado a un socket del lado servidor y, por tanto, tiene acceso a sus dos manejadores de flujo, `dos` y `dis`, que utilizará para comunicarse con su cliente, siguiendo los métodos ya conocidos. Lo más relevante de la clase `TCPServerThread` es la utilización de un recurso compartido por todos los hilos, que es accesible para cada hilo a través de su variable de instancia `serverState`. El objeto apuntado por esta variable ejecuta el método `fetchAndIncrementNumRequests` con cada nueva solicitud de datos del cliente. En este caso, el dato facilitado al cliente es la fecha y hora actual.

`TCPServerState.java` : recurso compartido entre los hilos del servidor

Aquí puede verse cómo el método que da acceso al recurso compartido, `fetchAndIncrementNumRequests`, ha sido declarado como `synchronized`, para así evitar que dos o más hilos lean o escriban al mismo tiempo en la variable privada `nextRequestNumber`. Esta variable guarda el número total de conexiones que han sido atendidas por el servidor.

```
public class TCPServerState {
    private int nextRequestNumber = 0;

    public synchronized int fetchAndIncrementNumRequests () {
        return ++nextRequestNumber;
    }
}
```

`TCPClient.java` : Cliente TCP

La implementación de la parte cliente es similar a la vista en el boletín anterior, ya que el cliente no sabe, ni tiene por qué saber, si al otro lado le está atendiendo el hilo principal del servidor o un hilo secundario.

Ejercicios propuestos

Planteamos esta semana que vuestro programa *NanoFiles* permita que un servidor lanzado mediante `serve` pueda atender simultáneamente conexiones de varios *peers*.

Con el fin de que dicho objetivo general pueda realizarse como una secuencia de pasos, os sugerimos una posible forma de proceder:

1. Implementa un servidor de ficheros concurrente en la clase `NFServer`, siguiendo las instrucciones que encontrarás en los *TODOs* del código proporcionado, y tomando como referencia el código del proyecto `boletinTCPconcurrente` que encontrarás en el Aula Virtual y ha sido descrito en este boletín. El método `run` de `NFServer` debería implementar el hilo principal del servidor concurrente que se va a ejecutar en segundo plano. Ya deberías haber programado el constructor de dicha clase en el boletín anterior. Ahora, debes añadir a la clase los métodos necesarios para crear un nuevo hilo que ejecute el método `run`, cuando se ejecute el comando `serve`.
2. Programa el lanzamiento del servidor en segundo plano en el método `startFileServer` de la clase `NFControllerLogicP2P.java`, siguiendo las instrucciones que encontrarás en los *TODOs* del código proporcionado. Este método se invoca cuando tecleas el comando `serve`, de forma que tras lanzar el servidor de ficheros en un hilo nuevo (en segundo plano), el hilo principal retorna y vuelve a ejecutar el código del *shell* de *NanoFiles* para seguir procesando comandos.
3. Programa el método `run`. Este será en parte similar al que has programado en el método `test` en lo que respecta a la puesta a la escucha de un servidor TCP en un determinado puerto. Tras aceptar la conexión de un cliente, el hilo principal del servidor debería crear un nuevo hilo para atender a ese cliente, de forma que el hilo secundario ejecute el código de `serveFilesToClient`.
4. Implementa también la clase `NFServerThread` (constructor y método `run`). Cada instancia (hilo) debe recibir el `Socket` aceptado por `NFServer` y delegar la atención del cliente en `NFServer.serveFilesToClient(socket)`.
5. Para que otros *peers* puedan localizar al servidor y descargar sus ficheros es necesario informar al directorio de que el par ahora está sirviendo ficheros. Para ello, deberías haber implementado ya el método `registerFileServer` de `DirectoryConnector`.
6. Por último, recuerda que el extremo servidor de la transferencia está en `NFServer.serveFilesToClient`, mientras que en el lado cliente la descarga se implementa en los métodos de `NFConnector` invocados desde `NFControllerLogicP2P.downloadFileFromServers`. Es en estos puntos donde habrás de crear, enviar y recibir los mensajes que has diseñado, modelados como objetos `PeerMessage`, así como leer/escribir fragmentos de ficheros.